

Raport optymalizacji zapytan - Indeksy SQL

Dokument przedstawia analize wydajnosci dwoch czesto wykonywanych zapytan SELECT przed i po dodaniu indeksow nieklastrowanych. Plany zapytan oraz statystyki I/O zostaly pobrane bezposrednio z silnika Microsoft SQL Server 2022 (SET SHOWPLAN_TEXT oraz SET STATISTICS IO).

Srodowisko testowe

Baza danych	Microsoft SQL Server 2022 (kontener Docker)
ORM / migracje	Entity Framework Core 10 (Code First)
Wolumen danych	Patients: 30 008 wierszy, Visits: 50 000 wierszy, lekarze: 200
Metoda pomiaru	SET SHOWPLAN_TEXT ON (operacja) + SET STATISTICS IO ON (odczyty logiczne)

Zapytanie nr 1 - wyszukiwanie pacjenta po numerze PESEL

Najczestsza operacja w module pacjentow (PatientService.SearchAsync) - filtrowanie po kolumnie niekluczowej Pesel:

```
SELECT Id, Pesel, FirstName, LastName
FROM Patients
WHERE Pesel = '00000012345';
```

PRZED: brak indeksu na kolumnie Pesel -> silnik przeszukuje cala tabele (Clustered Index Scan).

```
--Clustered Index Scan(OBJECT: Patients.PK_Patients,
WHERE: Patients.Pesel = N'00000012345')
```

PO: indeks nieklastrowany IX_Patients_Pesel -> bezposrednie wyszukanie (Index Seek).

```
--Index Seek(OBJECT: Patients.IX_Patients_Pesel,
SEEK: Patients.Pesel = N'00000012345' ORDERED FORWARD)
```

Metryka	PRZED (bez indeksu)	PO (z indeksem)
Operacja	Clustered Index Scan	Index Seek
Odczyty logiczne	421	4
Poprawa	~105x mniej odczytow logicznych, Scan -> Seek	

Zapytanie nr 2 - lista wizyt lekarza w danym dniu

Operacja grafiku lekarza - filtrowanie po DoctorId oraz zakresie VisitDate. Zoptymalizowana indeksem złożonym (DoctorId, VisitDate):

```
SELECT Id, PatientId, DoctorId, VisitDate
FROM Visits
WHERE DoctorId = 'doc-50'
      AND VisitDate >= '2026-03-15'
      AND VisitDate < '2026-03-16';
```

PRZED: brak pasującego indeksu -> Clustered Index Scan po 50 000 wierszy.

```
--Clustered Index Scan(OBJECT: Visits.PK_Visits,
  WHERE: VisitDate >= '2026-03-15' AND VisitDate < '2026-03-16'
        AND DoctorId = N'doc-50')
```

PO: indeks złożony IX_Visits_DoctorId_VisitDate -> Index Seek po obu kolumnach jednocześnie.

```
--Index Seek(OBJECT: Visits.IX_Visits_DoctorId_VisitDate,
  SEEK: DoctorId = N'doc-50'
        AND VisitDate >= '2026-03-15' AND VisitDate < '2026-03-16')
```

Metryka	PRZED (bez indeksu)	PO (z indeksem)
Operacja	Clustered Index Scan	Index Seek
Odczyty logiczne	374	3
Poprawa	~124x mniej odczytów logicznych, Scan -> Seek	

Wprowadzone zmiany w kodzie (EF Core)

Indeksy zdefiniowano deklaratywnie w ApplicationDbContext (Fluent API). Indeksy pojedyncze (Pesel, LastName, DoctorId, VisitDate) istniały już wcześniej; w ramach tego zadania dodano indeks złożony pod zapytanie nr 2:

```
builder.Entity<Visit>(entity =>
{
    entity.HasIndex(v => new { v.DoctorId, v.VisitDate })
        .HasDatabaseName("IX_Visits_DoctorId_VisitDate");
});
```

Zmiane wdrożono migracją EF Core:

```
dotnet ef migrations add AddIndexesForPerformance
dotnet ef database update
```

Podsumowanie

Dodanie indeksów nieklastrowanych zmieniło operacje dostępu do danych z kosztownego przeszukiwania całej tabeli (Table/Clustered Index Scan) na precyzyjne wyszukiwanie (Index Seek). Efekt mierzalny w liczbie odczytów logicznych:

Zapytanie 1 (PESEL) 421 -> 4 odczytów logicznych (~105x mniej)

Zapytanie 2 (lekarz+dzień) 374 -> 3 odczytów logicznych (~124x mniej)

Przy rosnącej liczbie rekordów różnica będzie się pogłębiać, ponieważ koszt operacji Scan rośnie liniowo z rozmiarem tabeli, podczas gdy koszt Index Seek pozostaje niemal stały (logarytmiczny).